



KUBERNETES

Hands-On Practice Guide

End-to-End · Debugging · Real-World Production

1 Master Node · 2 Worker Nodes · Production-Grade Exercises

Pods & Deployments · Services & Networking · Storage · RBAC · Helm · HPA · Taints & Tolerations · Node Affinity · Pod Affinity/Anti-Affinity · Priority Classes · Monitoring · Debugging · Real-World Scenarios

Table of Contents

1. Cluster Architecture & Setup Verification

- 1.1 Understanding Your 3-Node Cluster
- 1.2 Essential kubectl Commands

2. Core Kubernetes Objects – Deep Dive

- 2.1 Pods – The Atomic Unit
- 2.2 ReplicaSets
- 2.3 Deployments
- 2.4 Namespaces

3. Workload Management

- 3.1 Deployment Strategies (Rolling, Blue-Green, Canary)
- 3.2 StatefulSets
- 3.3 DaemonSets
- 3.4 Jobs & CronJobs

4. Services & Networking

- 4.1 All 5 Service Types – ClusterIP, NodePort, LoadBalancer, Headless, ExternalIP
- 4.2 Ingress Controllers
- 4.3 Network Policies – Ingress & Egress Rules
- 4.4 Pod Internet Access & Egress Troubleshooting
- 4.5 IP Blocking – Allowlists & Denylists
- 4.6 DNS Inside Kubernetes

5. Configuration & Secrets

- 5.1 ConfigMaps
- 5.2 Secrets
- 5.3 Environment Variables vs Volume Mounts

6. Storage

- 6.1 Volumes & PersistentVolumes
- 6.2 PersistentVolumeClaims
- 6.3 StorageClasses

7. RBAC & Security

- 7.1 ServiceAccounts
- 7.2 Roles & ClusterRoles
- 7.3 Pod Security

8. Helm – Package Management

- 8.1 Installing & Using Helm
- 8.2 Writing Your Own Chart

9. Auto-Scaling

- 9.1 Horizontal Pod Autoscaler (HPA)
- 9.2 Vertical Pod Autoscaler (VPA)
- 9.3 Cluster Autoscaler Concepts

10. Monitoring & Logging

- 10.1 Metrics Server
- 10.2 Prometheus + Grafana
- 10.3 Centralized Logging (EFK Stack)

11. Debugging Masterclass

- 11.1 Systematic Debugging Framework
- 11.2 Common Failure Scenarios & Fixes
- 11.3 Node-Level Debugging

12. Real-World Production Scenarios

- 12.1 Zero-Downtime Deployment
- 12.2 Database Migration with StatefulSet
- 12.3 Multi-Tier Application
- 12.4 Disaster Recovery Drill

13. Production Best Practices & Cheat Sheet

14. Scheduling, Placement & Workload Control

- 14.1 Taints & Tolerations
- 14.2 Node Affinity & Node Selector
- 14.3 Pod Affinity & Pod Anti-Affinity
- 14.4 Cordon, Drain & Node Maintenance
- 14.5 Priority Classes & Preemption
- 14.6 Pod Disruption Budgets (PDB)
- 14.7 Resource Quotas & LimitRanges
- 14.8 Topology Spread Constraints

1. Cluster Architecture & Setup Verification

Your cluster consists of one control-plane (master) node and two worker nodes. The control plane hosts the API server, etcd, scheduler, and controller manager. Worker nodes run the kubelet, kube-proxy, and your application workloads via containerd/docker.

1.1 Understanding Your 3-Node Cluster

	Role
kube-apiserver	Front door for all REST operations; validates and processes API objects
etcd	Distributed key-value store – single source of truth for cluster state
kube-scheduler	Assigns pods to nodes based on resource requests and constraints
kube-controller-manager	Runs controllers: ReplicaSet, Node, Job, etc.
kubelet	Node agent that runs on every node; ensures containers are running
kube-proxy	Maintains network rules on nodes for Service communication

1.2 Essential kubectl Commands

■ EXERCISE 1: Cluster Health Check

Step 1. Verify all nodes are Ready:

■ Shell

```
kubectl get nodes -o wide

# Expected output columns: NAME STATUS ROLES AGE VERSION INTERNAL-IP OS-IMAGE

kubectl describe node <node-name> # Deep inspect any node

kubectl top nodes # CPU/Memory (requires metrics-server)
```

Step 2. Check control-plane components:

■ Shell

```
kubectl get componentstatuses # Legacy but useful

kubectl get pods -n kube-system # All system pods

kubectl get pods -n kube-system -o wide --show-labels
```

Step 3. Explore the cluster info:

■ Shell

```
kubectl cluster-info

kubectl cluster-info dump | head -100 # Verbose cluster dump
```

```
kubectl api-resources # All available API resources
```

```
kubectl api-versions # All supported API versions
```

■ **TIP:** Always run 'kubectl get nodes' first thing when connecting to a cluster. If a node shows NotReady, investigate immediately with 'kubectl describe node '.

■ **NOTE:** Alias kubectl as 'k' in your shell: `echo 'alias k=kubectl' >> ~/.bashrc && source ~/.bashrc`

2. Core Kubernetes Objects – Deep Dive

2.1 Pods – The Atomic Unit

A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers that share network namespace and storage volumes. Pods are ephemeral — never rely on a Pod's IP or assume it will restart on the same node.

■ EXERCISE 2: Pod Lifecycle Exploration

Step 1. Create a simple nginx pod imperatively:

■ Shell

```
kubectl run nginx-pod --image=nginx:1.25 --port=80

kubectl get pod nginx-pod -w # Watch status changes live

kubectl get pod nginx-pod -o yaml # Full spec in YAML
```

Step 2. Inspect the pod in detail:

■ Shell

```
kubectl describe pod nginx-pod

# Look for: Events section, IP, Node, Conditions, Container state
```

Step 3. Create a multi-container pod (sidecar pattern):

■ YAML: multi-container-pod.yaml

```
apiVersion: v1

kind: Pod

metadata:
  name: app-with-sidecar

  labels:
    app: myapp
    tier: backend

spec:
  initContainers:
    - name: init-config
      image: busybox:1.35
      command: ['sh', '-c', 'echo Init complete > /shared/ready.txt']

  volumeMounts:
    - name: shared-data
```

```
mountPath: /shared

containers:

- name: main-app
  image: nginx:1.25
  ports:
  - containerPort: 80
  resources:
    requests:
      memory: "64Mi"
      cpu: "100m"
    limits:
      memory: "128Mi"
      cpu: "200m"
  volumeMounts:
  - name: shared-data
    mountPath: /usr/share/nginx/html
  livenessProbe:
    httpGet:
      path: /
      port: 80
    initialDelaySeconds: 10
    periodSeconds: 15
  readinessProbe:
    httpGet:
      path: /
      port: 80
    initialDelaySeconds: 5
    periodSeconds: 10

- name: log-sidecar
  image: busybox:1.35
  command: ['sh', '-c', 'while true; do date >> /shared/access.log; sleep 5; done']
```



```
volumeMounts:

- name: shared-data

mountPath: /shared

volumes:

- name: shared-data

emptyDir: {}
```

■ Apply & Test

```
kubectl apply -f multi-container-pod.yaml

kubectl get pod app-with-sidecar

kubectl logs app-with-sidecar -c main-app # Specific container logs

kubectl logs app-with-sidecar -c log-sidecar

kubectl exec -it app-with-sidecar -c main-app -- /bin/bash

kubectl exec -it app-with-sidecar -c log-sidecar -- cat /shared/access.log
```

■ **PRODUCTION:** Liveness probes restart containers that are stuck. Readiness probes remove pods from Service endpoints until they are healthy. Always configure both in production.

2.2 Deployments

Deployments manage ReplicaSets to maintain the desired number of Pod replicas. They enable declarative updates, rollbacks, and scaling.

■ EXERCISE 3: Deploy a Web Application

■ YAML: webapp-deployment.yaml

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: webapp

  namespace: default

  labels:

    app: webapp

    version: "1.0"

spec:

  replicas: 3

  selector:
```

```
matchLabels:

app: webapp

strategy:

type: RollingUpdate

rollingUpdate:

maxSurge: 1 # One extra pod during update

maxUnavailable: 0 # Zero downtime

template:

metadata:

labels:

app: webapp

version: "1.0"

spec:

affinity:

podAntiAffinity:

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 100

podAffinityTerm:

labelSelector:

matchExpressions:

- key: app

operator: In

values: [webapp]

topologyKey: kubernetes.io/hostname

containers:

- name: webapp

image: nginx:1.25

ports:

- containerPort: 80

resources:

requests:

memory: "64Mi"
```

```
cpu: "100m"

limits:

memory: "256Mi"

cpu: "500m"

env:

- name: APP_VERSION

value: "1.0"

- name: POD_NAME

valueFrom:

fieldRef:

fieldPath: metadata.name

- name: POD_NAMESPACE

valueFrom:

fieldRef:

fieldPath: metadata.namespace
```

■ Manage Deployment

```
kubectl apply -f webapp-deployment.yaml

kubectl rollout status deployment/webapp

kubectl get pods -l app=webapp -o wide # See distribution across nodes

# Scale up/down

kubectl scale deployment webapp --replicas=5

kubectl scale deployment webapp --replicas=2

# Update image

kubectl set image deployment/webapp webapp=nginx:1.26

kubectl rollout status deployment/webapp

# Rollback if something goes wrong

kubectl rollout history deployment/webapp

kubectl rollout undo deployment/webapp

kubectl rollout undo deployment/webapp --to-revision=2
```

3. Workload Management

3.1 Deployment Strategies

■ EXERCISE 4: Blue-Green Deployment

Deploy two identical environments. Switch traffic instantly by updating the Service selector.

■ YAML: [blue-green.yaml](#)

```
# Blue (current production)

apiVersion: apps/v1

kind: Deployment

metadata:

  name: webapp-blue

spec:

  replicas: 3

  selector:

    matchLabels:

      app: webapp

      slot: blue

  template:

    metadata:

      labels:

        app: webapp

        slot: blue

    spec:

      containers:

        - name: webapp

          image: nginx:1.25

---

# Green (new version - deploy, test, then switch)

apiVersion: apps/v1

kind: Deployment
```

```
metadata:
  name: webapp-green

spec:
  replicas: 3

  selector:
    matchLabels:
      app: webapp
      slot: green

  template:
    metadata:
      labels:
        app: webapp
        slot: green
    spec:
      containers:
        - name: webapp
          image: nginx:1.26
          ---
      apiVersion: v1
      kind: Service
      metadata:
        name: webapp-svc
      spec:
        selector:
          app: webapp
          slot: blue # <-- Change to 'green' to switch traffic
        ports:
          - port: 80
        targetPort: 80
```

■ Switch Traffic

```
# Test green deployment internally first
```

```
kubectl port-forward deployment/webapp-green 8080:80 &

curl localhost:8080

# Switch production traffic to green

kubectl patch service webapp-svc -p '{"spec":{"selector":{"slot":"green"}}}'

# Verify, then scale down blue

kubectl scale deployment webapp-blue --replicas=0
```

■ EXERCISE 5: Canary Deployment (5% Traffic)

■ YAML: canary.yaml

```
# Stable: 19 replicas = ~95% traffic

apiVersion: apps/v1

kind: Deployment

metadata:

  name: webapp-stable

  spec:

    replicas: 19

    selector:

      matchLabels:

        app: webapp

        track: stable

    template:

      metadata:

        labels:

          app: webapp

          track: stable

      spec:

        containers:

          - name: webapp

            image: nginx:1.25

          ---

# Canary: 1 replica = ~5% traffic
```

```
apiVersion: apps/v1

kind: Deployment

metadata:

name: webapp-canary

spec:

replicas: 1

selector:

matchLabels:

app: webapp

track: canary

template:

metadata:

labels:

app: webapp

track: canary

spec:

containers:

- name: webapp

image: nginx:1.26

---

apiVersion: v1

kind: Service

metadata:

name: webapp-svc

spec:

selector:

app: webapp # Selects BOTH stable and canary pods

ports:

- port: 80

targetPort: 80
```

3.2 StatefulSets

StatefulSets are for stateful applications (databases, queues) that need: stable network identity, ordered deployment/scaling, and persistent storage per pod.

■ EXERCISE 6: Deploy a 3-Replica MongoDB StatefulSet

■ YAML: `mongodb-statefulset.yaml`

```
apiVersion: apps/v1

kind: StatefulSet

metadata:

  name: mongodb

  namespace: databases

spec:

  serviceName: mongodb-headless

  replicas: 3

  selector:

    matchLabels:

      app: mongodb

  template:

    metadata:

      labels:

        app: mongodb

    spec:

      containers:

        - name: mongodb

          image: mongo:6.0

          ports:

            - containerPort: 27017

          env:

            - name: MONGO_INITDB_ROOT_USERNAME

              valueFrom:

                secretKeyRef:

                  name: mongodb-secret

                  key: username

            - name: MONGO_INITDB_ROOT_PASSWORD
```



```
valueFrom:

secretKeyRef:

name: mongodb-secret

key: password

volumeMounts:

- name: mongodb-data

mountPath: /data/db

resources:

requests:

memory: "512Mi"

cpu: "250m"

limits:

memory: "1Gi"

cpu: "500m"

volumeClaimTemplates:

- metadata:

name: mongodb-data

spec:

accessModes: ["ReadWriteOnce"]

storageClassName: standard

resources:

requests:

storage: 10Gi

---

apiVersion: v1

kind: Service

metadata:

name: mongodb-headless

namespace: databases

spec:

clusterIP: None # Headless service for StatefulSet

selector:
```

```
app: mongodb

ports:

- port: 27017
```

■ Observe StatefulSet Behaviour

```
# Pods get stable DNS names: mongodb-0, mongodb-1, mongodb-2

kubectl get pods -n databases -l app=mongodb

kubectl exec -it mongodb-0 -n databases -- mongo -u admin -p secret

# Scale and observe ordering (graceful shutdown from highest index)

kubectl scale statefulset mongodb --replicas=5 -n databases

kubectl scale statefulset mongodb --replicas=2 -n databases
```

■ **NOTE:** StatefulSet pods are DNS-addressable as: ...svc.cluster.local e.g. mongodb-0.mongodb-headless.databases.svc.cluster.local

3.3 DaemonSets

DaemonSets ensure one pod runs on every (or selected) node. Ideal for node-level agents: log collectors, monitoring agents, network plugins.

■ YAML: node-exporter-daemonset.yaml

```
apiVersion: apps/v1

kind: DaemonSet

metadata:

  name: node-exporter

  namespace: monitoring

spec:

  selector:

    matchLabels:

      app: node-exporter

  template:

    metadata:

      labels:

        app: node-exporter

    spec:

      hostNetwork: true
```

```
hostPID: true

tolerations:

- key: node-role.kubernetes.io/control-plane

operator: Exists

effect: NoSchedule # Run on master too

containers:

- name: node-exporter

image: prom/node-exporter:v1.6.1

ports:

- containerPort: 9100

hostPort: 9100

securityContext:

privileged: true

volumeMounts:

- name: sys

mountPath: /host/sys

readOnly: true

- name: root

mountPath: /host/root

readOnly: true

volumes:

- name: sys

hostPath:

path: /sys

- name: root

hostPath:

path: /
```

3.4 Jobs & CronJobs

■ YAML: db-backup-cronjob.yaml

```
apiVersion: batch/v1

kind: CronJob
```

```
metadata:

name: db-backup

namespace: databases

spec:

schedule: "0 2 * * *" # 2:00 AM daily

concurrencyPolicy: Forbid # Don't run if previous still running

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

jobTemplate:

spec:

backoffLimit: 3 # Retry up to 3 times on failure

template:

spec:

restartPolicy: OnFailure

containers:

- name: db-backup

image: postgres:15

command:

- /bin/sh

- -c

- |

pg_dump -h $DB_HOST -U $DB_USER $DB_NAME | gzip > /backup/backup-$(date
+%Y%m%d-%H%M%S).sql.gz

echo "Backup completed successfully"

env:

- name: DB_HOST

value: "postgres-service"

- name: DB_USER

valueFrom:

secretKeyRef:

name: postgres-secret

key: username
```

```
- name: PGPASSWORD

valueFrom:

secretKeyRef:

name: postgres-secret

key: password

- name: DB_NAME

value: "production_db"

volumeMounts:

- name: backup-storage

mountPath: /backup

volumes:

- name: backup-storage

persistentVolumeClaim:

claimName: backup-pvc
```



```
- protocol: TCP

port: 80

targetPort: 8080

externalIPs:

- 192.168.1.100 # Traffic to this IP:80 on any node is routed to the service

- 192.168.1.101 # Add multiple IPs if needed
```

■ Verify All Services

```
kubectl apply -f all-services.yaml

kubectl get svc -o wide

# ClusterIP - test from inside cluster only

kubectl run curl-test --image=curlimages/curl --rm -it --restart=Never -- curl
http://webapp-clusterip

# NodePort - test from outside (replace with actual node IP)

curl http://192.168.1.10:30080

curl http://192.168.1.11:30080 # Works on EVERY node

# LoadBalancer - get external IP then test

kubectl get svc webapp-lb --watch # Wait for EXTERNAL-IP to be assigned

curl https://<EXTERNAL-IP>

# Headless - DNS returns individual pod IPs (not a VIP)

kubectl run dns-test --image=busybox --rm -it --restart=Never -- nslookup
mongodb-headless.databases.svc.cluster.local

# Returns: mongodb-0.mongodb-headless... and mongodb-1.mongodb-headless...

# ExternalIP - test from a machine on the same network

curl http://192.168.1.100:80
```

■ **NOTE:** Headless services do NOT load-balance — they return all pod IPs via DNS and let the client decide. This is intentional for databases like Cassandra or MongoDB where the client needs to connect to specific nodes.

■■ **WARNING:** ExternalIP requires that the IP is actually routable to a node. It is NOT auto-provisioned like LoadBalancer. Use it when you have a static IP on a node NIC or a floating IP managed externally.

4.2 Ingress Controllers

Ingress provides HTTP/HTTPS routing to Services. An Ingress Controller (e.g., NGINX) must be installed to process Ingress resources.

■ Install NGINX Ingress Controller

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx

helm repo update

helm install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-nginx
--create-namespace --set controller.replicaCount=2

kubectl get pods -n ingress-nginx

kubectl get svc -n ingress-nginx
```

■ YAML: `ingress.yaml`

```
apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    cert-manager.io/cluster-issuer: letsencrypt-prod
  spec:
    ingressClassName: nginx
    tls:
      - hosts:
          - app.example.com
          - api.example.com
        secretName: app-tls-secret
    rules:
      - host: app.example.com
        http:
          paths:
            - path: /
              pathType: Prefix
          backend:
            service:
```

```
name: webapp-clusterip

port:

number: 80

- host: api.example.com

http:

paths:

- path: /v1

pathType: Prefix

backend:

service:

name: api-service

port:

number: 8080
```

4.3 Network Policies – Ingress & Egress Rules

Network Policies are the Kubernetes-native firewall. Without any policy, all pods can talk to all pods. Once you apply even one NetworkPolicy that selects a pod, that pod is isolated for the policy type(s) listed — all other traffic for that type is dropped.

■ **WARNING:** NetworkPolicy requires a CNI plugin that supports it (Calico, Cilium, Weave). Flannel alone does NOT enforce NetworkPolicy. Check your CNI before relying on policies.

■ EXERCISE 8: Default-Deny Then Selectively Allow

■ YAML: default-deny-all.yaml

```
# Apply this FIRST to any namespace you want to lock down.

# This drops ALL ingress and egress from every pod in the namespace.

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: default-deny-all

namespace: production

spec:

podSelector: {} # {} = matches ALL pods in the namespace

policyTypes:
```

- Ingress
- Egress

■ YAML: allow-frontend-to-backend.yaml

[illegible]

[illegible]

■■ **WARNING:** Forgetting to allow UDP/TCP port 53 (DNS) in egress rules is the #1 cause of mysterious pod failures after applying NetworkPolicy. Pods silently fail to resolve service names.

■ Verify Policy is Working

```
# Apply policies

kubectl apply -f default-deny-all.yaml

kubectl apply -f allow-frontend-to-backend.yaml


# Confirm frontend CAN reach backend

kubectl exec -n production deploy/frontend -- curl -s http://backend-svc:8080/health


# Confirm a random pod CANNOT reach backend (should timeout/refused)

kubectl run attacker --image=curlimages/curl --rm -it --restart=Never -n production
-- curl --max-time 5 http://backend-svc:8080
```

```
# List all NetworkPolicies

kubectl get networkpolicy -A

# Inspect which policies apply to a pod

kubectl describe pod <pod-name> -n production | grep -i network
```

4.4 Pod Internet Access & Egress Troubleshooting

Pods sometimes lose internet access due to NetworkPolicy egress rules, missing NAT/masquerade rules, or node-level firewall issues. This section gives you a systematic checklist to diagnose and fix it.

■ EXERCISE 9: Diagnose: Pod Has No Internet Access

■ Step-by-Step Diagnosis

```
# Step 1 - Confirm the problem is internet access (not DNS or internal)

kubectl exec -it <pod-name> -- curl --max-time 5 https://ifconfig.me

# If this hangs or fails, internet egress is blocked.

# Step 2 - Check if DNS works

kubectl exec -it <pod-name> -- nslookup google.com

# If DNS fails too, check if UDP 53 egress is allowed in NetworkPolicy.

# Step 3 - Check if the NODE has internet (SSH to node)

curl --max-time 5 https://ifconfig.me

# If node fails, it's a node-level or VPC routing issue - not Kubernetes.

# Step 4 - Check iptables masquerade rules on the node (must be root)

iptables -t nat -L POSTROUTING -n -v | grep MASQUERADE

# You should see a MASQUERADE rule for pod CIDR.

# If missing, kube-proxy or CNI masquerade is broken.

# Step 5 - Check if a NetworkPolicy is blocking egress

kubectl get networkpolicy -n <namespace>

kubectl describe networkpolicy <name> -n <namespace>

# Look for 'Egress' rules. If policyTypes includes Egress but no

# egress rules allow internet, traffic is dropped.

# Step 6 - Check node routing table
```

```
ip route show

# The default route (0.0.0.0/0) must point to the node's gateway.
```

■ YAML: allow-internet-egress.yaml

```
# Add this policy to allow pods to reach the internet while still
# blocking all unwanted ingress.

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-internet-egress
  namespace: production
spec:
  podSelector:
    matchLabels:
      internet-access: "true" # Only pods with this label get internet
  policyTypes:
    - Egress
  egress:
    # Allow DNS
    - ports:
        - protocol: UDP
          port: 53
      - protocol: TCP
          port: 53
    # Allow HTTPS to anywhere on the internet
    - ports:
        - protocol: TCP
          port: 443
    # Allow HTTP (remove in production if not needed)
    - ports:
        - protocol: TCP
          port: 80
```

■ Label Pods That Need Internet

```
# Add label to deployment pods

kubectl patch deployment my-app -p
'{"spec":{"template":{"metadata":{"labels":{"internet-access":"true"}}}}}'

# Verify the label was applied

kubectl get pods -n production --show-labels | grep internet-access

# Test internet from labelled pod

kubectl exec -it <my-app-pod> -- curl https://ifconfig.me
```

■ **PRODUCTION:** PRODUCTION TIP: Never grant internet egress to all pods. Use a label-based policy so only pods that explicitly need outbound internet access get it. This follows the principle of least privilege.

■ Common Egress Issues & Fixes

```
# Issue 1: Pod can reach cluster IPs but not internet

# Fix: Check iptables MASQUERADE rule exists on nodes

iptables -t nat -A POSTROUTING -s 10.244.0.0/16 ! -d 10.244.0.0/16 -j MASQUERADE

# Issue 2: DNS works but HTTPS fails

# Fix: Ensure port 443 TCP is in egress rules AND not blocked by node firewall

ufw status # Ubuntu

firewall-cmd --list-all # CentOS/RHEL

# Issue 3: Pod gets RESET (connection refused immediately, not timeout)

# Fix: Something actively blocking. Check cloud Security Groups / NSGs.

# Issue 4: Works on some nodes but not others

# Fix: Node-specific iptables or CNI misconfiguration.

# Check CNI pod on the failing node:

kubectl get pods -n kube-system -o wide | grep <node-name>

kubectl logs -n kube-system <cni-pod-on-bad-node>
```

4.5 IP Blocking – Allowlists & Denylists

Blocking or allowing specific external IP ranges can be done at three layers in Kubernetes: NetworkPolicy (pod level), LoadBalancer source ranges (service level), and Ingress annotations (HTTP level). Use all three for defense in depth.

■ EXERCISE 10: Block & Allow Specific IPs at Every Layer

■ Layer 1 – LoadBalancer: Restrict Source IPs (Service-Level)

```
# loadBalancerSourceRanges acts as an allowlist for a LoadBalancer service.
# Traffic from IPs NOT in this list is dropped at the cloud LB level.

apiVersion: v1

kind: Service

metadata:

  name: webapp-lb

spec:

  type: LoadBalancer

  selector:

    app: webapp

  ports:

    - port: 443

    targetPort: 8080

    loadBalancerSourceRanges:

      - "203.0.113.0/24" # Your office CIDR

      - "198.51.100.50/32" # A specific trusted IP

  # All other source IPs are blocked at the LB
```

■ Layer 2 – NetworkPolicy: Block / Allow by IP Block (Pod-Level)

```
# ipBlock lets you write rules for external CIDR ranges (outside the cluster).
# Use 'except' to carve out sub-ranges from an allowed block.

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

  name: restrict-external-ips

  namespace: production

spec:

  podSelector:

    matchLabels:

      app: webapp

  policyTypes:
```

```
- Ingress
- Egress

ingress:

# ALLOW traffic from trusted CIDR only

- from:

- ipBlock:

cidr: 203.0.113.0/24 # Allowed range

except:

- 203.0.113.50/32 # But block this specific IP inside the range

ports:

- protocol: TCP

port: 8080

egress:

# ALLOW egress to a specific third-party API server

- to:

- ipBlock:

cidr: 198.51.100.0/24 # Allowed external destination

ports:

- protocol: TCP

port: 443

# BLOCK egress to known bad CIDR (implicit: anything not listed is already blocked
# when policyTypes includes Egress and no matching rule exists)

# Always allow DNS

- to:

- namespaceSelector: {}

ports:

- protocol: UDP

port: 53
```

■ Layer 3 – NGINX Ingress: HTTP-Level IP Allowlist & Denylist

```
# Use NGINX annotations for HTTP-level IP filtering.

# This works AFTER the request reaches the ingress controller.

apiVersion: networking.k8s.io/v1
```

```
kind: Ingress

metadata:
  name: app-ingress

  annotations:

    # Allowlist: only these IPs can access this ingress rule
    nginx.ingress.kubernetes.io/whitelist-source-range:
      "203.0.113.0/24,198.51.100.50/32"

    # OR: use a ConfigMap-based denylist for blocking specific IPs
    # nginx.ingress.kubernetes.io/configuration-snippet: |
    # deny 192.0.2.100;
    # deny 192.0.2.200;
    # allow all;

spec:
  ingressClassName: nginx

  rules:

    - host: app.example.com

    http:

      paths:

        - path: /

        pathType: Prefix

      backend:

        service:

          name: webapp-clusterip

          port:

            number: 80
```

■ Verify IP Blocking is Working

```
# Test allowlisted IP CAN reach the service (run from allowed IP)

curl -v https://app.example.com

# Test blocked IP is rejected - simulate from inside cluster with a spoofed header

kubectl run block-test --image=curlimages/curl --rm -it --restart=Never -- curl -H
"X-Forwarded-For: 192.0.2.100" https://app.example.com

# Should return 403 Forbidden if NGINX whitelist is active
```

```
# Check NGINX ingress controller logs for blocked requests

kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=50 |
grep -E "403|deny|block"

# Verify NetworkPolicy ipBlock rules

kubectl describe networkpolicy restrict-external-ips -n production
```

■ **PRODUCTION:** Use all three layers together: LoadBalancer source ranges (cloud LB), NetworkPolicy ipBlock (kernel iptables), and Ingress annotations (NGINX). Each layer is independent, giving defense-in-depth. A pod-level NetworkPolicy protects even if the LB is misconfigured.

4.6 DNS Inside Kubernetes

CoreDNS runs as a Deployment in kube-system and handles all in-cluster name resolution. Every pod gets /etc/resolv.conf pointing to the CoreDNS ClusterIP.

■ DNS Name Formats

```
# Service DNS

<service>.<namespace>.svc.cluster.local

webapp-clusterip.default.svc.cluster.local

# Headless pod DNS (StatefulSet)

<pod-name>.<headless-service>.<namespace>.svc.cluster.local

mongodb-0.mongodb-headless.databases.svc.cluster.local

# Short forms that work within the same namespace

curl http://webapp-clusterip # same namespace

curl http://webapp-clusterip.default # cross-namespace short form
```

■ Troubleshoot DNS

```
# Check CoreDNS pods are running

kubectl get pods -n kube-system -l k8s-app=kube-dns

# Run a DNS debug pod

kubectl run dns-debug --image=busybox --rm -it --restart=Never -- sh

> nslookup webapp-clusterip.default.svc.cluster.local

> nslookup kubernetes.default.svc.cluster.local # should always work

> cat /etc/resolv.conf

# Check CoreDNS logs for errors
```

```
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50

# Edit CoreDNS ConfigMap to add custom DNS entries

kubectl edit configmap coredns -n kube-system
```

■ **TIP:** Always test cluster DNS with 'nslookup kubernetes.default.svc.cluster.local' from a pod. If this fails, CoreDNS is broken — not your app. Fix CoreDNS before debugging anything else.

5. Configuration & Secrets

5.1 ConfigMaps

■ EXERCISE 8: ConfigMap – All Three Patterns

■ YAML: configmap.yaml

```
apiVersion: v1

kind: ConfigMap

metadata:

  name: app-config

  namespace: default

  data:

    # Key-value pairs

    DATABASE_URL: "postgres://postgres-svc:5432/appdb"

    LOG_LEVEL: "INFO"

    MAX_CONNECTIONS: "100"

    # Entire file as a value

    app.properties: |

      server.port=8080

      server.timeout=30s

      feature.dark_mode=true

      feature.beta_api=false

    nginx.conf: |

      server {

        listen 80;

        location / {

          proxy_pass http://backend:8080;

          proxy_set_header Host $host;

        }

      }
```

■ Use in Pod – Three Methods

```
# Method 1: Environment variables from ConfigMap

spec:

containers:

- name: app

envFrom:

- configMapRef:

name: app-config

# Method 2: Specific key as env var

env:

- name: LOG_LEVEL

valueFrom:

configMapKeyRef:

name: app-config

key: LOG_LEVEL

# Method 3: Volume mount (hot-reload without restart!)

volumeMounts:

- name: config-volume

mountPath: /etc/config

volumes:

- name: config-volume

configMap:

name: app-config
```

5.2 Secrets

■■ **WARNING:** Kubernetes Secrets are base64 encoded, NOT encrypted by default. Enable Encryption at Rest or use Sealed Secrets / Vault in production.

■ Create Secrets

```
# Imperative (values are base64 auto-encoded)

kubectl create secret generic db-secret --from-literal=username=admin
--from-literal=password='S3cur3P@ss!'

# From file (useful for TLS certs)
```

```
kubectl create secret tls webapp-tls --cert=tls.crt --key=tls.key

# Docker registry secret

kubectl create secret docker-registry regcred --docker-server=registry.example.com
--docker-username=user --docker-password=pass --docker-email=user@example.com
```

■ YAML: secret.yaml (manual base64)

```
apiVersion: v1

kind: Secret

metadata:

name: db-secret

type: Opaque

data:

username: YWRtaW4= # echo -n 'admin' | base64

password: UzNjdXIzUEBzcyE= # echo -n 'S3cur3P@ss!' | base64
```


6. Storage – PVs, PVCs & StorageClasses

Kubernetes decouples storage provisioning from consumption. PersistentVolumes (PV) represent actual storage. PersistentVolumeClaims (PVC) are requests for storage. StorageClasses enable dynamic provisioning.

■ EXERCISE 9: Full Storage Setup – Static & Dynamic

■ YAML: storage.yaml

```
# StorageClass – dynamic provisioning

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

  name: fast-ssd

  annotations:

    storageclass.kubernetes.io/is-default-class: "false"

  provisioner: rancher.io/local-path # or kubernetes.io/aws-ebs, etc.

  reclaimPolicy: Retain

  allowVolumeExpansion: true

  volumeBindingMode: WaitForFirstConsumer

---

# PersistentVolume – static (for NFS/local storage)

apiVersion: v1

kind: PersistentVolume

metadata:

  name: local-pv-node1

  labels:

    type: local

  spec:

    storageClassName: fast-ssd

    capacity:

      storage: 20Gi

    accessModes:

      - ReadWriteOnce
```

```
persistentVolumeReclaimPolicy: Retain

local:

path: /mnt/disks/ssd1

nodeAffinity:

  required:

    nodeSelectorTerms:

      - matchExpressions:

          - key: kubernetes.io/hostname

            operator: In

            values: [worker-node-1]

      ---

# PersistentVolumeClaim - application request

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

  name: app-data-pvc

  namespace: default

spec:

  storageClassName: fast-ssd

  accessModes:

    - ReadWriteOnce

  resources:

    requests:

      storage: 5Gi

  ---

# Pod using the PVC

apiVersion: v1

kind: Pod

metadata:

  name: app-with-storage

spec:

  containers:
```

```
- name: app

image: nginx:1.25

volumeMounts:

- name: app-data

mountPath: /var/app/data

volumes:

- name: app-data

persistentVolumeClaim:

claimName: app-data-pvc
```

■ Inspect Storage

```
kubectl get pv,pvc

kubectl describe pv local-pv-node1

kubectl describe pvc app-data-pvc

# Check binding status: PVC should show Bound

# Expand PVC (requires allowVolumeExpansion: true)

kubectl patch pvc app-data-pvc -p
'{"spec":{"resources":{"requests":{"storage":"10Gi"}}}}'
```

■ **PRODUCTION:** ReclaimPolicy: 'Retain' keeps data after PVC deletion (safe for production). 'Delete' removes the storage. 'Recycle' is deprecated.

7. RBAC & Security

Role-Based Access Control (RBAC) is Kubernetes' authorization system. It controls who can do what on which resources. Following least-privilege is critical in production.

■ EXERCISE 10: Create a Developer Role with Restricted Access

■ YAML: `rbac.yaml`

```
# ServiceAccount for the application

apiVersion: v1

kind: ServiceAccount

metadata:

  name: webapp-sa

  namespace: production

---

# Role - namespace-scoped

apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:

  name: developer-role

  namespace: production

rules:

- apiGroups: [""]

  resources: ["pods", "pods/log", "services", "configmaps"]

  verbs: ["get", "list", "watch"]

- apiGroups: ["apps"]

  resources: ["deployments", "replicasets"]

  verbs: ["get", "list", "watch", "create", "update", "patch"]

- apiGroups: [""]

  resources: ["pods/exec"]

  verbs: ["create"] # Allow exec into pods

---

# RoleBinding

apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: RoleBinding

metadata:

name: developer-binding

namespace: production

subjects:

- kind: User

name: developer-alice

apiGroup: rbac.authorization.k8s.io

- kind: ServiceAccount

name: webapp-sa

namespace: production

roleRef:

kind: Role

name: developer-role

apiGroup: rbac.authorization.k8s.io

---

# ClusterRole for read-only cluster-wide access

apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRole

metadata:

name: cluster-viewer

rules:

- apiGroups: [""]

resources: ["nodes", "namespaces", "persistentvolumes"]

verbs: ["get", "list", "watch"]

- apiGroups: ["metrics.k8s.io"]

resources: ["nodes", "pods"]

verbs: ["get", "list"]
```

■ Test RBAC Permissions

```
# Check what a user CAN do

kubectl auth can-i list pods --namespace=production --as=developer-alice
```

```
kubectl auth can-i delete pods --namespace=production --as=developer-alice # Should
return: no

# List all permissions for a service account

kubectl auth can-i --list --namespace=production
--as=system:serviceaccount:production:webapp-sa

# Audit: who can do what

kubectl get rolebindings,clusterrolebindings -A | grep developer
```

7.3 Pod Security Standards

■ YAML: secure-pod.yaml

```
apiVersion: v1

kind: Pod

metadata:

name: secure-pod

spec:

securityContext:

runAsNonRoot: true

runAsUser: 1000

runAsGroup: 3000

fsGroup: 2000

seccompProfile:

type: RuntimeDefault

containers:

- name: app

image: nginx:1.25

securityContext:

allowPrivilegeEscalation: false

readOnlyRootFilesystem: true

capabilities:

drop:

- ALL

add:
```

```
- NET_BIND_SERVICE

volumeMounts:

- name: tmp

mountPath: /tmp

- name: var-cache

mountPath: /var/cache/nginx

- name: var-run

mountPath: /var/run

volumes:

- name: tmp

emptyDir: {}

- name: var-cache

emptyDir: {}

- name: var-run

emptyDir: {}
```

■ **PRODUCTION:** Enable Pod Security Admission on namespaces: `kubectl label namespace production pod-security.kubernetes.io/enforce=restricted`

8. Helm – Package Management

Helm is the package manager for Kubernetes. Charts are reusable templates that package all K8s manifests for an application.

■ EXERCISE 11: Create & Deploy a Custom Helm Chart

■ Create Chart Scaffold

```
helm create myapp

# Creates:

# myapp/

# Chart.yaml – Chart metadata

# values.yaml – Default configuration values

# templates/ – Kubernetes manifest templates

# templates/NOTES.txt – Post-install notes

# charts/ – Chart dependencies
```

■ Chart.yaml

```
apiVersion: v2

name: myapp

description: Production-grade web application

type: application

version: 0.1.0

appVersion: "1.0.0"

maintainers:

- name: Platform Team

  email: platform@example.com
```

■ values.yaml (defaults)

```
replicaCount: 3

image:

repository: nginx

tag: "1.25"

pullPolicy: IfNotPresent

service:

type: ClusterIP
```



```
port: 80

ingress:

enabled: true

host: app.example.com

resources:

requests:

cpu: 100m

memory: 128Mi

limits:

cpu: 500m

memory: 256Mi

autoscaling:

enabled: true

minReplicas: 2

maxReplicas: 10

targetCPUUtilizationPercentage: 70
```

■ Helm Lifecycle Commands

```
# Lint and dry-run

helm lint myapp/

helm template myapp myapp/ --values prod-values.yaml # Preview rendered YAML

helm install myapp ./myapp --namespace production --create-namespace

helm upgrade myapp ./myapp --values prod-values.yaml --atomic --timeout 5m

helm rollback myapp 1 # Rollback to revision 1

helm history myapp

helm uninstall myapp -n production

# Override values at install time

helm install myapp ./myapp --set replicaCount=5 --set image.tag=1.26 --set
ingress.host=newapp.example.com
```

9. Auto-Scaling

9.1 Horizontal Pod Autoscaler (HPA)

HPA automatically scales pod replicas based on CPU/memory or custom metrics.

■ EXERCISE 12: Configure HPA with Custom Metrics

■ Install Metrics Server (prerequisite)

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Verify

kubectl top pods

kubectl top nodes
```

■ YAML: hpa.yaml

```
apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

  name: webapp-hpa

  namespace: production

spec:

  scaleTargetRef:

    apiVersion: apps/v1

    kind: Deployment

    name: webapp

  minReplicas: 2

  maxReplicas: 20

  metrics:

    - type: Resource

      resource:

        name: cpu

        target:

          type: Utilization

          averageUtilization: 70
```

```
- type: Resource

resource:

  name: memory

  target:

    type: Utilization

    averageUtilization: 80

  behavior:

    scaleUp:

      stabilizationWindowSeconds: 60

    policies:

      - type: Pods

        value: 4

        periodSeconds: 60 # Add max 4 pods per minute

    scaleDown:

      stabilizationWindowSeconds: 300 # 5 min cooldown before scaling down
```

■ Simulate Load & Watch HPA

```
# Generate load

kubectl run load-gen --image=busybox --rm -it --restart=Never -- sh -c 'while true;
do wget -q -O- http://webapp-clusterip; done'

# Watch HPA in real time (separate terminal)

kubectl get hpa webapp-hpa -w

kubectl describe hpa webapp-hpa # Detailed scaling events
```

■■ **WARNING:** Always define resource requests on your pods! HPA cannot calculate utilization percentage without them.

9.2 Pod Disruption Budgets

PodDisruptionBudgets guarantee availability during voluntary disruptions (node drain, upgrades).

■ YAML: `pdb.yaml`

```
apiVersion: policy/v1

kind: PodDisruptionBudget

metadata:

  name: webapp-pdb
```

```
spec:  
  
  minAvailable: 2 # OR maxUnavailable: 1  
  
  selector:  
  
    matchLabels:  
  
      app: webapp
```

10. Monitoring & Logging

10.1 Prometheus + Grafana Stack

■ Install kube-prometheus-stack via Helm

```
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts

helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack --namespace
monitoring --create-namespace --set prometheus.prometheusSpec.retention=15d --set
prometheus.prometheusSpec.retentionSize=10GB --set alertmanager.enabled=true --set
grafana.adminPassword=admin123

kubectl get pods -n monitoring

kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring

# Open http://localhost:3000 (admin / admin123)
```

■ Custom PrometheusRule Alert

```
apiVersion: monitoring.coreos.com/v1

kind: PrometheusRule

metadata:

  name: webapp-alerts

  namespace: monitoring

  labels:

    release: monitoring

  spec:

    groups:

      - name: webapp

        interval: 30s

        rules:

          - alert: WebappHighErrorRate

            expr: |

              rate(http_requests_total{job="webapp",status=~"5.."}[5m])

              / rate(http_requests_total{job="webapp"}[5m]) > 0.01

            for: 5m

            labels:
```

```
severity: critical

annotations:

summary: "High error rate on {{ $labels.instance }}"

description: "Error rate is {{ $value | humanizePercentage }}"

- alert: PodCrashLooping

expr: rate(kube_pod_container_status_restarts_total[15m]) * 60 > 0

for: 5m

labels:

severity: warning

annotations:

summary: "Pod {{ $labels.namespace }}/{{ $labels.pod }} is crash looping"
```

10.2 Centralized Logging (EFK)

The EFK stack (Elasticsearch + Fluentd + Kibana) provides centralized log aggregation across all nodes.

■ Fluentd DaemonSet Config Snippet

```
# fluentd collects logs from /var/log/containers/ on each node

# and forwards to Elasticsearch

<source>

@type tail

path /var/log/containers/*.log

pos_file /var/log/fluentd-containers.log.pos

tag kubernetes.*

read_from_head true

<parse>

@type multi_format

<pattern>

format json

time_key time

keep_time_key true

</pattern>

</parse>
```

```
</source>

<filter kubernetes.**>

@type kubernetes_metadata

</filter>

<match kubernetes.**>

@type elasticsearch

host elasticsearch-service

port 9200

logstash_format true

logstash_prefix k8s-logs

<buffer>

flush_interval 5s

retry_max_times 3

</buffer>

</match>
```

11. Debugging Masterclass

Systematic debugging is the most critical production skill. Follow the hierarchy: Pod → Node → Network → Storage → Control Plane.

11.1 Systematic Debugging Framework

	<code>kubectl get pod -n</code>	Identify the phase: Pending, Running, CrashLoopBackOff, Error, OOMKilled, Evicted
2. Pod Events	<code>kubectl describe pod</code>	Check Events section at bottom – most errors show up here first
3. Container Logs	<code>kubectl logs -c</code>	Current logs. Add --previous for last crashed container logs
4. Exec In	<code>kubectl exec -it -- /bin/sh</code>	Run commands inside the container. Check env vars, file system, network
5. Node Health	<code>kubectl describe node</code>	Check Conditions, Allocatable resources, Taints
6. Events Cluster-wide	<code>kubectl get events --sort-by=.lastTimestamp</code>	All events sorted by time. Excellent first overview

11.2 Common Failure Scenarios & Fixes

Scenario: CrashLoopBackOff

■ Debug Commands & Fix

```
# Pod keeps restarting

kubectl logs <pod> --previous # See why it crashed

kubectl describe pod <pod> # Check exit code in Last State

# Common causes:

# - Application error (check logs for stack trace)

# - Misconfigured env vars or secrets

# - Missing readiness probe grace period

# - OOMKilled (increase memory limits)

# Fix: kubectl set resources deploy <name> --limits=memory=512Mi
```

Scenario: ImagePullBackOff / ErrImagePull

■ Debug Commands & Fix

```
kubectl describe pod <pod> # See exact pull error
```



```
# Common causes:

# - Wrong image name/tag

# - Private registry without imagePullSecret

# - Network issue pulling from registry

# Fix - add imagePullSecret:

kubectl create secret docker-registry regcred \

--docker-server=registry.example.com \

--docker-username=user --docker-password=pass

# Reference in pod spec:

# imagePullSecrets:

# - name: regcred
```

Scenario: Pending Pods (Insufficient Resources)

■ Debug Commands & Fix

```
kubectl describe pod <pod> # Look for: Insufficient cpu/memory

kubectl get nodes # Check if nodes are Ready

kubectl describe nodes | grep -A5 "Allocated resources"

# Check resource requests vs available

kubectl top nodes

# Fix options:

# 1. Lower resource requests in pod spec

# 2. Add more nodes

# 3. Check for taints: kubectl describe node | grep Taint

# 4. Add tolerations to pod spec if tainted
```

Scenario: Service Not Routing Traffic

■ Debug Commands & Fix

```
# Debug service connectivity

kubectl get endpoints <svc-name> # Should show pod IPs, not <none>

kubectl describe svc <svc-name>

# If endpoints is empty = label selector mismatch!
```

```
kubectl get pods --show-labels

# Compare pod labels with svc selector

# Test from inside cluster

kubectl run debug --image=curlimages/curl --rm -it --restart=Never \

-- curl http://<svc-name>.<namespace>.svc.cluster.local

# Check kube-proxy logs

kubectl logs -n kube-system -l k8s-app=kube-proxy
```

Scenario: PVC Stuck in Pending

■ Debug Commands & Fix

```
kubectl describe pvc <pvc-name>

# Check for: no persistent volumes available for this claim

kubectl get pv # Any available PVs?

kubectl get storageclass # Is default StorageClass set?

# Common fixes:

# 1. Create matching PV (static provisioning)

# 2. Install dynamic provisioner (local-path, AWS EBS, etc.)

# 3. Check access modes match (ReadWriteOnce vs ReadWriteMany)

# 4. Check storage size (PVC request <= PV capacity)
```

Scenario: Node NotReady

■ Debug Commands & Fix

```
kubectl describe node <node-name> # Check Conditions section

kubectl get pods -n kube-system --field-selector spec.nodeName=<node>

# SSH into the node

ssh user@<node-ip>

systemctl status kubelet # Is kubelet running?

journalctl -u kubelet -n 100 # Kubelet logs

systemctl restart kubelet # Restart if crashed

df -h # Disk full? (common cause)

free -h # Memory pressure?
```

```
systemctl status containerd # Container runtime OK?
```

11.3 Ephemeral Debug Containers

■ Debug Production Pods Safely

```
# Add debug container to running pod (K8s 1.23+)

kubectl debug -it <pod> --image=busybox --target=<container>

# Create debug copy of a crashing pod

kubectl debug <pod> -it --copy-to=debug-pod --image=busybox

# Debug a node directly

kubectl debug node/<node-name> -it --image=ubuntu

# Network debugging toolkit

kubectl run netshoot --rm -it --image=nicolaka/netshoot -- bash

# Inside netshoot: tcpdump, dig, curl, nmap, iperf3 all available
```

12. Real-World Production Scenarios

12.1 Scenario: Zero-Downtime Deployment Pipeline

This scenario simulates a production deployment with pre-deployment checks, rolling update, smoke tests, and automatic rollback.

■ EXERCISE 13: Full Production Deployment Workflow

■ deploy.sh – Production Deploy Script

```
#!/bin/bash

set -euo pipefail

NAMESPACE="production"

DEPLOYMENT="webapp"

NEW_IMAGE="myapp:2.1.0"

TIMEOUT="5m"

echo "=== Pre-deployment health check ==="

kubectl get nodes | grep -v NotReady || { echo "FAIL: Unhealthy nodes"; exit 1; }

echo "=== Saving current revision for rollback ==="

CURRENT_REVISION=$(kubectl rollout history deployment/$DEPLOYMENT -n $NAMESPACE
--no-headers | tail -1 | awk '{print $1}')

echo "Current revision: $CURRENT_REVISION"

echo "=== Deploying $NEW_IMAGE ==="

kubectl set image deployment/$DEPLOYMENT webapp=$NEW_IMAGE -n $NAMESPACE

kubectl annotate deployment/$DEPLOYMENT kubernetes.io/change-cause="Deploy
$NEW_IMAGE by $USER at $(date)" -n $NAMESPACE

echo "=== Waiting for rollout ==="

if ! kubectl rollout status deployment/$DEPLOYMENT -n $NAMESPACE --timeout=$TIMEOUT;
then

echo "FAIL: Rollout failed! Rolling back to revision $CURRENT_REVISION"

kubectl rollout undo deployment/$DEPLOYMENT -n $NAMESPACE
--to-revision=$CURRENT_REVISION

exit 1

fi
```

```
echo "=== Smoke test ==="

POD=$(kubectl get pod -n $NAMESPACE -l app=webapp -o
jsonpath='{.items[0].metadata.name}')

RESPONSE=$(kubectl exec $POD -n $NAMESPACE -- curl -s -o /dev/null -w "%{http_code}"
http://localhost:80/health)

if [ "$RESPONSE" != "200" ]; then

echo "FAIL: Health check returned $RESPONSE - rolling back"

kubectl rollout undo deployment/$DEPLOYMENT -n $NAMESPACE

exit 1

fi

echo "SUCCESS: Deployment of $NEW_IMAGE completed!"

kubectl rollout history deployment/$DEPLOYMENT -n $NAMESPACE
```

12.2 Scenario: Multi-Tier Application

Deploy a complete 3-tier application: React frontend, Node.js API, PostgreSQL database — all with proper separation of concerns.

■ EXERCISE 14: Deploy Full-Stack Application

■ Namespace Structure

```
kubectl create namespace frontend

kubectl create namespace backend

kubectl create namespace databases

# Label namespaces for Network Policies

kubectl label namespace frontend name=frontend

kubectl label namespace backend name=backend

kubectl label namespace databases name=databases
```

■ YAML: full-stack-app.yaml (key excerpts)

```
# === DATABASE TIER ===

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: postgres

namespace: databases
```

```
spec:
  replicas: 1
  serviceName: postgres
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
        tier: database
    spec:
      containers:
        - name: postgres
          image: postgres:15
          env:
            - name: POSTGRES_DB
              value: "appdb"
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: password
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data
          volumeClaimTemplates:
            - metadata:
                name: postgres-data
              spec:
```

```
accessModes: ["ReadWriteOnce"]

resources:

requests:

storage: 20Gi

---

# === BACKEND TIER ===

apiVersion: apps/v1

kind: Deployment

metadata:

name: api-server

namespace: backend

spec:

replicas: 3

selector:

matchLabels:

app: api-server

template:

metadata:

labels:

app: api-server

tier: backend

spec:

serviceAccountName: api-sa

containers:

- name: api

image: mycompany/api:1.0

ports:

- containerPort: 8080

env:

- name: DB_HOST

value: "postgres.databases.svc.cluster.local"

- name: DB_PASSWORD
```

```
valueFrom:

secretKeyRef:

name: postgres-secret

key: password

readinessProbe:

httpGet:

path: /api/health

port: 8080

initialDelaySeconds: 10

periodSeconds: 5

---

# === FRONTEND TIER ===

apiVersion: apps/v1

kind: Deployment

metadata:

name: frontend

namespace: frontend

spec:

replicas: 2

selector:

matchLabels:

app: frontend

template:

metadata:

labels:

app: frontend

tier: frontend

spec:

containers:

- name: frontend

image: mycompany/frontend:1.0

ports:
```



```
- containerPort: 3000

env:

- name: API_URL

value: "http://api-server.backend.svc.cluster.local:8080"
```

12.3 Scenario: Disaster Recovery Drill

■ EXERCISE 15: Simulate and Recover from Node Failure

■ Disaster Recovery Steps

```
# === SIMULATE NODE FAILURE ===

# Cordon (prevent new pods) then drain (evict existing pods)

kubectl cordon worker-node-2

kubectl drain worker-node-2 --ignore-daemonsets --delete-emptydir-data
--grace-period=30

# Watch pods reschedule to healthy nodes

kubectl get pods -A -o wide -w

# === RECOVERY ===

# Once node is repaired/replaced

kubectl uncordon worker-node-2

kubectl get nodes

# Verify all pods are healthy

kubectl get pods -A | grep -v Running | grep -v Completed

# === ETCD BACKUP (run on master node) ===

ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%Y%m%d).db
--endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt
--cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt
--key=/etc/kubernetes/pki/etcd/healthcheck-client.key

# Verify backup

ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-$(date +%Y%m%d).db
--write-out=table

# === ETCD RESTORE ===
```

```
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-20240115.db
--data-dir=/var/lib/etcd-restored --name=master
--initial-cluster=master=https://127.0.0.1:2380
--initial-advertise-peer-urls=https://127.0.0.1:2380
```

■ **PRODUCTION:** Schedule etcd backups daily via CronJob on the master node. Keep at least 7 days of backups in an external storage system (S3, NFS). Test restores monthly.

13. Production Best Practices & Cheat Sheet

Production Checklist

✓ Resource Requests & Limits	ALWAYS set both on every container. Requests affect scheduling; limits prevent OOM.
✓ Health Probes	Configure liveness AND readiness probes. Set appropriate initialDelaySeconds.
✓ Anti-Affinity Rules	Use podAntiAffinity to spread pods across nodes for HA.
✓ Pod Disruption Budgets	Define PDBs for all stateful and critical services.
✓ Namespaces	Separate environments (dev/staging/prod) and teams using namespaces.
✓ Network Policies	Default-deny all ingress/egress. Whitelist only required paths.
✓ Secrets Management	Use Sealed Secrets or Vault. Never commit raw secrets to Git.
✓ Image Tags	Never use :latest in production. Pin to specific digest or immutable tag.
✓ RBAC	One ServiceAccount per application. Least-privilege access only.
✓ Resource Quotas	Set namespace-level ResourceQuotas to prevent resource hogging.
✓ Taints & Tolerations	Use taints to reserve nodes for specific workloads (GPU, infra, prod).
✓ Node Affinity	Prefer or require scheduling to specific node types using labels.
✓ Topology Spread	Use TopologySpreadConstraints to balance pods across zones/nodes.
✓ Priority Classes	Define PriorityClasses so critical pods preempt low-priority ones.
✓ Monitoring	Deploy Prometheus + Grafana. Set up alerting for critical metrics.
✓ Logging	Centralize logs with EFK or Loki+Grafana. Never rely on pod logs alone.
✓ etcd Backup	Automated daily backups. Test restore quarterly.
✓ Node Maintenance	Always cordon+drain before maintenance. Never force-delete pods.
✓ Image Scanning	Scan images with Trivy/Snyk in CI/CD before deploying.

Essential kubectl Cheat Sheet

# CONTEXT & CLUSTER	
<code>kubectl config get-contexts</code>	List all configured contexts / clusters
<code>kubectl config use-context <name></code>	Switch active cluster context
<code>kubectl config set-context --current --namespace=<ns></code>	Set default namespace for current context
<code>kubectl config view --minify</code>	Show only current context config
<code>kubectl cluster-info</code>	API server and CoreDNS addresses

<code>kubectl api-resources --verbs=list --namespaced</code>	All listable namespaced resources
<code>kubectl api-versions</code>	All supported API group versions
# VIEWING RESOURCES	
<code>kubectl get all -n <ns></code>	All core resources in a namespace
<code>kubectl get pods -A -o wide</code>	All pods cluster-wide with node + IP info
<code>kubectl get pods --sort-by=.metadata.creationTimestamp</code>	Sort pods by creation time (oldest first)
<code>kubectl get pods --sort-by=.status.phase</code>	Sort pods by phase (Pending/Running etc.)
<code>kubectl get pod <name> -o jsonpath='{.status.podIP}'</code>	Extract a specific field with JSONPath
<code>kubectl get pod <name> -o jsonpath='{.spec.nodeName}'</code>	Which node is this pod on?
<code>kubectl get pods -l app=webapp,env=prod</code>	Filter by multiple labels
<code>kubectl get nodes --show-labels</code>	Show all node labels
<code>kubectl get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints</code>	Show node taints
<code>kubectl get events --sort-by=.lastTimestamp -n <ns></code>	Recent events newest-last
<code>kubectl get events --field-selector reason=OOMKilling</code>	Filter events by reason
<code>kubectl top pods -A --sort-by=memory</code>	All pods sorted by memory usage
<code>kubectl top nodes</code>	CPU and memory per node
# DEBUGGING PODS	
<code>kubectl logs <pod> -f --tail=100</code>	Follow last 100 log lines
<code>kubectl logs <pod> --previous</code>	Logs from last (crashed) container run
<code>kubectl logs <pod> -c <container></code>	Logs from specific container in multi-container pod
<code>kubectl logs -l app=webapp --prefix</code>	Logs from all pods matching label
<code>kubectl exec -it <pod> -- /bin/sh</code>	Shell into a running pod
<code>kubectl exec -it <pod> -c <container> -- /bin/bash</code>	Shell into specific container
<code>kubectl port-forward svc/<svc> 8080:80</code>	Forward local port to a service
<code>kubectl port-forward pod/<pod> 9090:9090</code>	Forward local port directly to pod
<code>kubectl describe pod <pod> grep -A20 Events</code>	Show pod events only
<code>kubectl describe pod <pod> grep -i taint</code>	Check taint-related scheduling issues
<code>kubectl run debug --image=busybox --rm -it --restart=Never -- sh</code>	Throw-away debug pod
<code>kubectl run debug --image=nicolaka/netshoot --rm -it --restart=Never -- bash</code>	Full network debug pod
<code>kubectl debug node/<node> -it --image=busybox</code>	Debug at node level (k8s 1.23+)
# TAINTS, TOLERATIONS & AFFINITY	
<code>kubectl taint nodes <node> key=value:NoSchedule</code>	Add a taint to a node
<code>kubectl taint nodes <node> key=value:NoSchedule-</code>	Remove a taint (note trailing -)
<code>kubectl taint nodes <node> key=value:NoExecute</code>	Evict existing pods + block new ones
<code>kubectl describe node <node> grep -A5 Taints</code>	View taints on a node
<code>kubectl get pods -o wide grep <node></code>	All pods currently on a specific node

<code>kubectl label node <node> disktype=ssd</code>	Add a label to a node
<code>kubectl label node <node> disktype-</code>	Remove a node label
# NODE MANAGEMENT	
<code>kubectl cordon <node></code>	Mark node unschedulable (no new pods)
<code>kubectl uncordon <node></code>	Re-enable scheduling on a node
<code>kubectl drain <node> --ignore-daemonsets --delete-emissary-data</code>	Safely evict all pods from node
<code>kubectl drain <node> --ignore-daemonsets --force --grace-period=30</code>	Drain with force + grace period
<code>kubectl get nodes -o wide</code>	Node IPs, OS, kernel version
<code>kubectl describe node <node> grep -A10 Allocatable</code>	See allocatable CPU/mem on node
<code>kubectl describe node <node> grep -A30 'Non-terminated Pods'</code>	Pods using resources on node
# ROLLOUTS & DEPLOYMENTS	
<code>kubectl rollout status deployment/<name></code>	Watch rollout progress
<code>kubectl rollout history deployment/<name></code>	List all revision history
<code>kubectl rollout history deployment/<name> --revision=3</code>	Details of a specific revision
<code>kubectl rollout undo deployment/<name></code>	Rollback to previous revision
<code>kubectl rollout undo deployment/<name> --to-revision=2</code>	Rollback to specific revision
<code>kubectl rollout pause deployment/<name></code>	Pause a rollout mid-way
<code>kubectl rollout resume deployment/<name></code>	Resume a paused rollout
<code>kubectl scale deployment <name> --replicas=5</code>	Manually scale a deployment
<code>kubectl autoscale deployment <name> --min=2 --max=10 --cpu-percent=70</code>	Create HPA imperatively
# EDITING & PATCHING	
<code>kubectl edit deployment <name></code>	Live edit resource in \$EDITOR
<code>kubectl patch deploy <n> -p '{"spec":{"replicas":5}}'</code>	JSON patch a field
<code>kubectl patch node <n> -p '{"spec":{"unschedulable":true}}'</code>	Patch node to unschedulable
<code>kubectl apply --dry-run=client -f file.yaml</code>	Client-side dry run (no API call)
<code>kubectl apply --dry-run=server -f file.yaml</code>	Server-side dry run (validates on API)
<code>kubectl diff -f file.yaml</code>	Show what would change before applying
<code>kubectl set image deployment/<name> <c>=nginx:1.26</code>	Update container image
<code>kubectl set resources deploy/<name> -c=<c> --limits=cpu=500m,memory=512Mi</code>	Update resource limits live
<code>kubectl annotate pod <pod> note='debug session'</code>	Add annotation to a resource
# NAMESPACES & RBAC	
<code>kubectl create namespace staging</code>	Create a namespace
<code>kubectl get rolebindings,clusterrolebindings -A grep <user></code>	Find RBAC bindings for a user
<code>kubectl auth can-i create pods --as=system:serviceaccount:default:mysa</code>	Check if SA can do action
<code>kubectl auth can-i '*' '*' --all-namespaces</code>	Check if current user is cluster-admin
<code>kubectl create token <sa-name> -n <ns></code>	Generate a SA token (k8s 1.24+)

# RESOURCE QUOTAS & LIMITS	
<code>kubectl describe resourcequota -n <ns></code>	Show quota usage vs limits in namespace
<code>kubectl describe limitrange -n <ns></code>	Show default limits and ranges
<code>kubectl get pdb -A</code>	List all PodDisruptionBudgets
<code>kubectl describe pdb <name> -n <ns></code>	Check PDB status and disruptions allowed
<code>kubectl get priorityclass</code>	List all PriorityClasses
# STORAGE	
<code>kubectl get pv,pvc -A</code>	List all PVs and PVCs cluster-wide
<code>kubectl describe pvc <name> -n <ns></code>	Check PVC binding status and events
<code>kubectl get storageclass</code>	List storage classes
<code>kubectl patch pvc <name> -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'</code>	Expand PVC size
# NETWORK & SERVICES	
<code>kubectl get svc -A -o wide</code>	All services with selectors
<code>kubectl get endpoints <svc></code>	Show actual pod IPs backing a service
<code>kubectl get ingress -A</code>	List all ingress rules
<code>kubectl get networkpolicy -A</code>	List all network policies
<code>kubectl run curl --image=curlimages/curl --rm -it --restart=Never -- curl <url></code>	One-shot curl from inside cluster
# CLEANUP	
<code>kubectl delete pod <name> --grace-period=0 --force</code>	Force delete a stuck/terminating pod
<code>kubectl delete all -l app=webapp</code>	Delete all resources with a label
<code>kubectl delete pods --field-selector status.phase=Failed -A</code>	Remove all Failed pods cluster-wide
<code>kubectl delete pods --field-selector status.phase=Succeeded -A</code>	Remove all Completed pods cluster-wide
<code>kubectl delete evicted_pods() { kubectl get pods -A grep Evicted awk '{print \$2, \$1}' xargs -n2 sh -c 'kubectl delete pod \$0 -n \$1'; }</code>	Delete all Evicted pods (shell function)
# HELPFUL ALIASES & TRICKS	
<code>alias k=kubectl</code>	Short alias (add to ~/.bashrc)
<code>alias kgp='kubectl get pods -o wide'</code>	Quick pod listing alias
<code>alias kns='kubectl config set-context --current --namespace'</code>	Quick namespace switch
<code>export KUBE_EDITOR=nano</code>	Set preferred editor for kubectl edit
<code>kubectl get pods -w</code>	Watch pods update in real time
<code>kubectl explain pod.spec.affinity</code>	Built-in field documentation
<code>kubectl explain deployment.spec.strategy --recursive</code>	Recursive field docs

■ **Keep Practicing!** The best way to master Kubernetes is to break things intentionally — delete pods, drain nodes, exhaust resources, misconfigure services — then fix them. Every failure in a practice environment is a lesson that prevents an incident in production.

14. Scheduling, Placement & Workload Control

Kubernetes gives you fine-grained control over WHERE pods run, HOW they tolerate node conditions, and WHAT happens when nodes are under pressure. Mastering these mechanisms is essential for production reliability, multi-tenant clusters, and efficient resource utilization.

14.1 Taints & Tolerations

A **taint** is applied to a NODE — it repels pods from being scheduled there. A **toleration** is applied to a POD — it allows the pod to be scheduled onto a tainted node. Think of taints as 'this node is reserved' and tolerations as 'this pod has the key to enter'.

	New pods WITHOUT a matching toleration will NOT be scheduled on this node. Existing pods are NOT affected.
PreferNoSchedule	Kubernetes TRIES to avoid placing pods without a toleration here, but will do so if no alternatives exist.
NoExecute	New pods without toleration are NOT scheduled AND existing pods without toleration are EVICTED immediately (or after tolerationSeconds).

■ EXERCISE 14: Taints & Tolerations – Reserve a Node for Production

■ Taint Commands

```
# Add a taint to worker-node-1 so only prod pods can land on it
kubectl taint nodes worker-node-1 env=production:NoSchedule

# Verify the taint was applied
kubectl describe node worker-node-1 | grep -A5 Taints

# Add a NoExecute taint to immediately evict non-tolerating pods
kubectl taint nodes worker-node-1 env=production:NoExecute

# Remove a taint (append '-' to the taint key:effect)
kubectl taint nodes worker-node-1 env=production:NoSchedule-
kubectl taint nodes worker-node-1 env=production:NoExecute-

# Common built-in taints (applied automatically by Kubernetes)
# node.kubernetes.io/not-ready:NoExecute -- node is not ready
# node.kubernetes.io/unreachable:NoExecute -- node is unreachable
# node.kubernetes.io/disk-pressure:NoSchedule -- disk is full
# node.kubernetes.io/memory-pressure:NoSchedule -- memory is low
```

```
# node.kubernetes.io/unschedulable:NoSchedule -- kubectl cordon applied
```

■ YAML: toleration-examples.yaml

```
apiVersion: v1

kind: Pod

metadata:

  name: prod-pod

  spec:

    tolerations:

      # Exact match toleration - must match key, operator, value, effect

      - key: "env"

        operator: "Equal"

        value: "production"

        effect: "NoSchedule"

      # Exists toleration - matches any value for the key

      - key: "dedicated"

        operator: "Exists"

        effect: "NoSchedule"

      # Tolerate NoExecute with a time limit (pod evicted after 300s)

      - key: "node.kubernetes.io/not-ready"

        operator: "Exists"

        effect: "NoExecute"

        tolerationSeconds: 300

      # Wildcard: tolerate ALL taints (use with extreme caution in production)

      # - operator: "Exists"

    containers:

      - name: app

        image: nginx:1.25

      ---

      # Real-world: GPU node reserved for ML workloads

      # Step 1: taint the GPU node
```



```
# kubectl taint nodes gpu-node-1 nvidia.com/gpu=present:NoSchedule

# Step 2: add toleration to ML deployment

apiVersion: apps/v1

kind: Deployment

metadata:

  name: ml-training

spec:

  replicas: 2

  selector:

    matchLabels:

      app: ml-training

  template:

    metadata:

      labels:

        app: ml-training

    spec:

      tolerations:

        - key: "nvidia.com/gpu"

          operator: "Equal"

          value: "present"

          effect: "NoSchedule"

      containers:

        - name: trainer

          image: tensorflow/tensorflow:latest-gpu

          resources:

            limits:

              nvidia.com/gpu: 1
```

■ **NOTE:** Taints alone do NOT guarantee a pod lands on the tainted node — they only repel others. To FORCE a pod onto a specific node, combine taints with `nodeAffinity` or `nodeSelector`.

14.2 Node Affinity & nodeSelector

nodeSelector is the simple form: match exact node labels. **nodeAffinity** is the advanced form: supports operators (In, NotIn, Exists, Gt, Lt), hard requirements (requiredDuringScheduling) and soft preferences (preferredDuringScheduling).

■ EXERCISE 15: Schedule Pods to Specific Node Types

■ Label Nodes First

```
# Label your nodes to describe their hardware/role

kubectl label node worker-node-1 disktype=ssd zone=us-east-1a role=compute

kubectl label node worker-node-2 disktype=hdd zone=us-east-1b role=storage


# Verify labels

kubectl get nodes --show-labels

kubectl get nodes -l disktype=ssd
```

■ YAML: node-affinity.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ssd-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ssd-app
  template:
    metadata:
      labels:
        app: ssd-app
    spec:
      # ■■ Simple nodeSelector (exact match only)
      # nodeSelector:
      #   disktype: ssd
      # ■■ nodeAffinity (preferred over nodeSelector)
      affinity:
        nodeAffinity:
```

```
# HARD requirement: pod will NOT schedule if not met

requiredDuringSchedulingIgnoredDuringExecution:

nodeSelectorTerms:

- matchExpressions:

- key: disktype

operator: In # In, NotIn, Exists, DoesNotExist, Gt, Lt

values:

- ssd

- key: kubernetes.io/arch

operator: In

values:

- amd64

# SOFT preference: scheduler prefers this but won't block if not met

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 80 # Higher weight = stronger preference (1-100)

preference:

matchExpressions:

- key: zone

operator: In

values:

- us-east-1a

- weight: 20

preference:

matchExpressions:

- key: role

operator: In

values:

- compute

containers:

- name: app

image: nginx:1.25
```

■ Verify Scheduling

```
kubectl apply -f node-affinity.yaml

kubectl get pods -o wide # Check which nodes pods landed on

# Simulate: what if no node matches the hard requirement?

kubectl label node worker-node-1 disktype=hdd --overwrite

kubectl delete pod -l app=ssd-app

kubectl get pods # Pods will stay Pending - check events

kubectl describe pod <pod-name> | grep -A10 Events

# You'll see: "0/2 nodes matched node affinity"

# Restore

kubectl label node worker-node-1 disktype=ssd --overwrite
```

■ **NOTE:** IgnoredDuringExecution in the affinity name means: if a node's labels change AFTER a pod is already running, the pod is NOT evicted. A future 'RequiredDuringExecution' mode will evict pods — watch for it in newer k8s versions.

14.3 Pod Affinity & Pod Anti-Affinity

While node affinity controls which NODES pods prefer, pod affinity/anti-affinity controls the relationship between PODS themselves. **podAffinity:** schedule this pod NEAR other pods (co-locate). **podAntiAffinity:** schedule this pod AWAY from other pods (spread out).

■ EXERCISE 16: Co-locate and Spread Pods Strategically

■ YAML: pod-affinity-antiaffinity.yaml

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: webapp

spec:

  replicas: 3

  selector:

    matchLabels:

      app: webapp

  template:

    metadata:
```

```
labels:

app: webapp

spec:

affinity:

# ■■ POD ANTI-AFFINITY: spread webapp pods across nodes ■■■■■■■■■■

podAntiAffinity:

# HARD: never place 2 webapp pods on the same node

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

matchExpressions:

- key: app

operator: In

values: [webapp]

topologyKey: kubernetes.io/hostname # "hostname" = node-level spread

# SOFT: prefer to spread across availability zones too

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 100

podAffinityTerm:

labelSelector:

matchLabels:

app: webapp

topologyKey: topology.kubernetes.io/zone

# ■■ POD AFFINITY: place webapp near its cache pods ■■■■■■■■■■

podAffinity:

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 50

podAffinityTerm:

labelSelector:

matchLabels:

app: redis-cache # Co-locate with redis for low latency

topologyKey: kubernetes.io/hostname
```

```
containers:

- name: webapp

image: nginx:1.25

---

# Cache deployment that webapp wants to be near

apiVersion: apps/v1

kind: Deployment

metadata:

name: redis-cache

spec:

replicas: 2

selector:

matchLabels:

app: redis-cache

template:

metadata:

labels:

app: redis-cache

spec:

containers:

- name: redis

image: redis:7
```

■ Verify Spreading

```
kubectl apply -f pod-affinity-antiaffinity.yaml

# Check webapp pods are on DIFFERENT nodes (anti-affinity)

kubectl get pods -l app=webapp -o wide

# Check webapp pods are on SAME nodes as redis (affinity)

kubectl get pods -l app=redis-cache -o wide

# If anti-affinity blocks scheduling (more replicas than nodes):

kubectl get pods # You'll see Pending pods
```

```
kubectl describe pod <pending-pod> | grep -A15 Events

# Event: "didn't match pod affinity/anti-affinity rules"
```

■ ■ **WARNING:** Hard `podAntiAffinity` with `topologyKey=hostname` means you can never have more replicas than nodes. Use 'preferred' for >3 replicas across a 2-node cluster, or you'll get stuck Pending pods.

14.4 Cordon, Drain & Node Maintenance

Before any node maintenance (OS upgrade, hardware swap, kernel patch), you must safely move workloads off it. The `cordon` → `drain` → `maintain` → `uncordon` workflow ensures zero disruption to running services.

■ EXERCISE 17: Safe Node Maintenance Workflow

■ Full Maintenance Procedure

```
# STEP 1: Cordon - mark node unschedulable (no NEW pods land here)

kubectl cordon worker-node-2

kubectl get nodes # STATUS shows: Ready,SchedulingDisabled


# STEP 2: Drain - evict all running pods gracefully

# --ignore-daemonsets: DaemonSet pods are NOT evicted (they run everywhere by design)

# --delete-emissary-data: removes local data for emissary pods

kubectl drain worker-node-2 --ignore-daemonsets --delete-emissary-data
--grace-period=60 --timeout=300s


# If drain is blocked by a PodDisruptionBudget (PDB):

kubectl drain worker-node-2 --ignore-daemonsets --disable-eviction=true

# WARNING: --disable-eviction bypasses PDBs - use only in emergencies


# STEP 3: Verify node is empty

kubectl get pods -o wide | grep worker-node-2

# Should show no pods (except DaemonSet pods)


# STEP 4: Perform maintenance on the node (SSH, patch, reboot, etc.)

# ssh admin@worker-node-2

# sudo apt upgrade -y && sudo reboot


# STEP 5: Uncordon - allow new pods to schedule here again

kubectl uncordon worker-node-2
```

```
kubectl get nodes # STATUS returns to: Ready

# STEP 6: Verify new pods can land on it

kubectl get pods -o wide # Eventually pods should spread back
```

■■ **WARNING:** Always check for PodDisruptionBudgets before draining: 'kubectl get pdb -A'. A PDB with minAvailable=100% will block drain entirely until you scale up first.

14.5 Priority Classes & Preemption

PriorityClasses define the relative importance of pods. When the cluster is resource-constrained, the scheduler can EVICT (preempt) lower-priority pods to make room for higher-priority ones. Critical for ensuring production workloads always have resources.

■ EXERCISE 18: Create Priority Classes and Observe Preemption

■ **YAML:** [priority-classes.yaml](#)

```
# Define priority tiers for your organization

apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:
  name: critical-production
  value: 1000000 # Higher number = higher priority
  globalDefault: false
  preemptionPolicy: PreemptLowerPriority # Will evict lower-priority pods
  description: "For mission-critical production services. Preempts all others."
---

apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:
  name: high-priority
  value: 100000
  globalDefault: false
  preemptionPolicy: PreemptLowerPriority
  description: "Important services. Preempts default and low priority."
---
```



```
apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:

name: default-priority

value: 1000

globalDefault: true # All pods without a priorityClassName get this

preemptionPolicy: PreemptLowerPriority

description: "Default for all workloads unless specified."

---

apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:

name: low-priority-batch

value: 100

globalDefault: false

preemptionPolicy: Never # Will not preempt others - just waits

description: "Batch/background jobs. Yields resources when cluster is busy."

---

# Apply to a deployment

apiVersion: apps/v1

kind: Deployment

metadata:

name: payment-service

spec:

replicas: 2

selector:

matchLabels:

app: payment-service

template:

metadata:

labels:

app: payment-service
```

```
spec:

priorityClassName: critical-production # <-- assign priority here

containers:

- name: payment

image: my-payment-service:v2

resources:

requests:

cpu: "500m"

memory: "512Mi"

limits:

cpu: "1000m"

memory: "1Gi"
```

■ Observe Preemption

```
kubectl apply -f priority-classes.yaml

kubectl get priorityclass

# Simulate resource pressure: fill the cluster with low-priority batch pods

kubectl create deployment batch-filler --image=nginx --replicas=20

kubectl get pods # batch-filler pods running, consuming resources

# Now deploy a critical pod (will preempt batch pods)

kubectl create deployment critical-app --image=nginx

kubectl patch deployment critical-app -p
'{"spec":{"template":{"spec":{"priorityClassName":"critical-production"}}}}'

# Watch preemption happen

kubectl get pods -w

# You will see batch-filler pods move to Terminating as critical-app gets scheduled

# Check which pod was preempted and why

kubectl get events --sort-by=.lastTimestamp | grep -i preempt
```

■■ **WARNING:** Kubernetes system components (kube-dns, metrics-server) already use built-in PriorityClasses: system-cluster-critical (2000000000) and system-node-critical (2000001000). Never assign these to your own workloads.

14.6 Pod Disruption Budgets (PDB)

A PodDisruptionBudget guarantees a minimum number of pods remain available during voluntary disruptions (drain, rolling update, node maintenance). It is your safety net against accidental full-service outages during operations.

■ EXERCISE 19: Protect Services with PDBs

■ YAML: `pdb-examples.yaml`

```
# PDB using minAvailable: at least 2 pods must always be running

apiVersion: policy/v1

kind: PodDisruptionBudget

metadata:

  name: webapp-pdb

  namespace: production

spec:

  minAvailable: 2 # Absolute count - keep at least 2 running

  selector:

    matchLabels:

      app: webapp

  ---

# PDB using maxUnavailable: at most 1 pod can be down at a time

apiVersion: policy/v1

kind: PodDisruptionBudget

metadata:

  name: api-pdb

  namespace: production

spec:

  maxUnavailable: 1 # At most 1 pod can be unavailable

  selector:

    matchLabels:

      app: api-service

  ---

# PDB using percentage

apiVersion: policy/v1
```

```
kind: PodDisruptionBudget

metadata:

name: worker-pdb

spec:

minAvailable: "50%" # At least 50% must be available

selector:

matchLabels:

app: worker
```

■ Verify PDB is Working

```
kubectl apply -f pdb-examples.yaml

# Check PDB status

kubectl get pdb -n production

# Output columns: NAME MIN-AVAILABLE MAX-UNAVAILABLE ALLOWED-DISRUPTIONS AGE

# ALLOWED-DISRUPTIONS = how many pods can currently be disrupted

kubectl describe pdb webapp-pdb -n production

# Test: try to drain a node that would violate the PDB

kubectl drain worker-node-1 --ignore-daemonsets

# If draining would take webapp below minAvailable=2, drain will BLOCK with:

# "Cannot evict pod as it would violate the pod's disruption budget"

# Check which pods are blocking

kubectl get pod -l app=webapp -o wide # See how many are on the node being drained

kubectl get pdb webapp-pdb -n production -o jsonpath='{.status.disruptionsAllowed}'
```

■ **PRODUCTION:** Always set PDBs BEFORE any maintenance. A PDB of minAvailable=1 with 3 replicas lets you drain one node at a time safely. Without a PDB, drain can take all pods down simultaneously.

14.7 Resource Quotas & LimitRanges

ResourceQuota limits the total resources a namespace can consume. **LimitRange** sets default and maximum resources per individual pod/container. Together they prevent any one team or namespace from starving others.

■ EXERCISE 20: Enforce Resource Governance Per Namespace

■ **YAML: resource-governance.yaml**

```
# Namespace-level quota: total caps across all pods in the namespace

apiVersion: v1

kind: ResourceQuota

metadata:

  name: production-quota

  namespace: production

spec:

  hard:

    # Compute

    requests.cpu: "10" # Total CPU requests across all pods

    requests.memory: "20Gi" # Total memory requests

    limits.cpu: "20" # Total CPU limits

    limits.memory: "40Gi" # Total memory limits

    # Object count limits

    pods: "50" # Max 50 pods

    services: "20"

    services.loadbalancers: "3" # Max 3 LoadBalancer services ($$)

    persistentvolumeclaims: "15"

    secrets: "30"

    configmaps: "30"

    ---

  # LimitRange: per-container defaults and caps

  apiVersion: v1

  kind: LimitRange

  metadata:

    name: container-limits

    namespace: production

  spec:

    limits:

      - type: Container

    default: # Applied when a container has NO limits set
```

```

cpu: "200m"

memory: "256Mi"

defaultRequest: # Applied when a container has NO requests set

cpu: "100m"

memory: "128Mi"

max: # Hard ceiling per container

cpu: "2"

memory: "4Gi"

min: # Minimum allowed

cpu: "50m"

memory: "64Mi"

- type: Pod

max: # Hard ceiling per entire pod

cpu: "4"

memory: "8Gi"

- type: PersistentVolumeClaim

max:

storage: "50Gi"

min:

storage: "1Gi"

```

■ Verify Quota Enforcement

```

# See current usage vs limits

kubectl describe resourcequota production-quota -n production

# Output shows: Resource | Used | Hard

# See LimitRange defaults

kubectl describe limitrange container-limits -n production

# Test: try to create a pod that exceeds quota

kubectl run big-pod -n production --image=nginx --requests='cpu=100'
--limits='cpu=200'

# Error: exceeded quota: production-quota

# Test: create a pod with no resources - LimitRange injects defaults

```

```
kubectl run default-pod -n production --image=nginx

kubectl describe pod default-pod -n production | grep -A6 Limits

# Will show the default limits injected by LimitRange
```

■ **NOTE:** If a pod fails to schedule with 'exceeded quota' but quota seems to have headroom, check that the pod's requests match what's in the quota. LimitRange defaults are only applied at admission time — not retroactively.

14.8 Topology Spread Constraints

TopologySpreadConstraints give precise control over how pods are distributed across topology domains (zones, nodes, racks). They supersede podAntiAffinity for spreading use cases and offer more flexibility with the maxSkew parameter.

■ EXERCISE 21: Evenly Spread Pods Across Nodes and Zones

■ YAML: topology-spread.yaml

```
apiVersion: apps/v1

kind: Deployment

metadata:
  name: webapp-spread

spec:
  replicas: 6

  selector:
    matchLabels:
      app: webapp-spread

  template:
    metadata:
      labels:
        app: webapp-spread

    spec:
      topologySpreadConstraints:

        # Constraint 1: spread evenly across nodes (hostname)
        - maxSkew: 1 # Max difference between most/least loaded domain

        topologyKey: kubernetes.io/hostname

      whenUnsatisfiable: DoNotSchedule # Hard: block scheduling if constraint violated
```

```
labelSelector:

matchLabels:

app: webapp-spread

# Constraint 2: spread across availability zones (soft)

- maxSkew: 1

topologyKey: topology.kubernetes.io/zone

whenUnsatisfiable: ScheduleAnyway # Soft: try but don't block

labelSelector:

matchLabels:

app: webapp-spread

containers:

- name: webapp

image: nginx:1.25
```

■ Verify Spread

```
kubectl apply -f topology-spread.yaml

# With 6 replicas on 2 nodes, expect 3 pods per node

kubectl get pods -l app=webapp-spread -o wide

# Check skew: count pods per node

kubectl get pods -l app=webapp-spread -o wide | awk '{print $7}' | sort | uniq -c

# Label your nodes with zone info (required for zone spreading)

kubectl label node worker-node-1 topology.kubernetes.io/zone=us-east-1a

kubectl label node worker-node-2 topology.kubernetes.io/zone=us-east-1b

# After labelling, delete and recreate pods to trigger re-spread

kubectl rollout restart deployment webapp-spread
```

	At most 1 pod difference between the most-loaded and least-loaded domain
DoNotSchedule	Hard constraint — pod stays Pending if constraint cannot be satisfied
ScheduleAnyway	Soft constraint — scheduler picks the node that minimises skew but won't block
topologyKey	The node label used to define 'domains' (hostname=node, zone=AZ, rack=rack)

■ **TIP:** TopologySpreadConstraints work WITH your replica count. For 3 nodes and replicas=3 with maxSkew=1, you get perfect 1-1-1 distribution. For replicas=4, you get 2-1-1 which still satisfies maxSkew=1.

■ **PRODUCTION:** PRODUCTION PATTERN: Use TopologySpreadConstraints instead of podAntiAffinity for spreading. TSC is more flexible (maxSkew), works with any topology domain, and gives better scheduler performance at scale.

Scheduling Decision Quick-Reference

Use this table to pick the right mechanism for your scheduling goal:

Reserve a node for specific workloads only	Taint + Toleration (NoSchedule)
Immediately evict pods from a node	Taint (NoExecute)
Run pods only on nodes with specific hardware	nodeAffinity (required)
Prefer certain nodes but allow fallback	nodeAffinity (preferred) with weights
Co-locate two services for low latency	podAffinity (preferred, same hostname)
Spread replicas across all nodes	podAntiAffinity OR TopologySpreadConstraints
Spread across zones for HA	TopologySpreadConstraints (zone topologyKey)
Ensure critical pods always get resources	PriorityClass (high value, PreemptLowerPriority)
Protect service during node drain	PodDisruptionBudget (minAvailable)
Cap resource usage per namespace	ResourceQuota
Set default limits per container	LimitRange

■ **You now have the full scheduling toolkit.** Combine taints+tolerations for node isolation, node/pod affinity for placement preferences, topology spread for even distribution, priority classes for resource contention, and PDBs+quotas for safety guardrails. These are the skills that separate a Kubernetes operator from a Kubernetes engineer.